

Lung Nodule Segmentation — SegResNet 3D — WIDE, v2 split

1. Task

Voxel-level segmentation of pulmonary nodules in 3D chest CT volumes, cropped to a per-series lung bounding box.

- **Input** (1, 256, 256, 256) CT crop, intensity-normalised to [0, 1], resampled from the bbox (padding = 20 vox).
- **Output** (2, 256, 256, 256) softmax logits — class 0 = background, class 1 = nodule.

2. Model

[MONAI SegResNet](#), SegResNet (residual U-Net).

Property	Value
Architecture family	SegResNet (residual U-Net)
Spatial dims	3
Input channels	1
Output channels	2 (softmax)
Initial feature width	32 (wide — 4× the small variant)
Encoder blocks per level	[1, 2, 2, 4, 4]
Decoder blocks per level	[1, 1, 1, 1]
Dropout	0.1
Trainable parameters	82,637,282

3. Data

Trained on the **unified_v2** split (patient-grouped, dataset-stratified, **full corpus**: NLST + NSCLC + LIDC-IDRI).

Split sizes: **1 683 train / 297 val / 325 test (held out)**.

Note on bboxes. This model was originally trained with the newer `bboxes_unified_v2.json` bboxes (tighter lung crops, regenerated), which differ from the `processed/bboxes_unified.json` bundled with this demo. For byte-for-byte reproduction of the reported test-set numbers, use the v2 bboxes; otherwise expect small variance from the ~62 % of series whose bbox coordinates differ.

Class imbalance: nodule voxels are on the order of $\sim 10^{-5}$ of the total, which shapes how each metric should be read (§7).

4. Training

Setting	Value
Loss	Focal Tversky + weighted CE ($\alpha=0.3$, $\beta=0.7$, $\gamma=2.0$, $\lambda_{CE}=0.1-0.3$, <code>ce_nodule_weight = 100</code>)
Optimizer	Adam
Learning rate	1×10^{-5}
Weight decay	1×10^{-5}
LR schedule	Cosine annealing, $T_{max} = 1000$, $\eta_{min} = 1 \times 10^{-6}$
Batch size	2
Epochs	1000
Random seed	42
Mixed precision	bf16 (autocast, no GradScaler)
Augmentation	3D flips, 90° rotations, elastic rotation, zoom, intensity scale/shift, Gaussian noise/blur, contrast
Hardware	1 × NVIDIA H100 94 GB
Wall-clock	≈ 12 days

Best checkpoint (evaluated below): **epoch 489 / 1000**.

5. Evaluation protocol

Held-out test split (325 series). The training loop used the `val` split of `unified_v2` for early stopping and best-Dice checkpoint selection; the numbers below come from the completely untouched **test** split (never seen during training or model selection). Predictions taken as `argmax` over the 2-channel softmax output; foreground = class 1. All metrics are computed at the voxel level and micro-averaged over the whole test split (5,452,595,200 voxels total).

6. Results

Ticket-required metrics

Metric	Value
mean IoU	0.7482
Accuracy	0.9996

Metric	Value
Precision	0.5766
Recall	0.7818

Supplementary

Metric	Value
IoU (foreground / nodule class)	0.4967
IoU (background class)	0.9996
Dice / F1 (micro)	0.6637

Per-case Dice distribution (325 val cases)

Statistic	Value
Mean	0.5706
Std	0.2598
Min	0.0000
p05	0.0000
p25	0.4069
Median	0.6304
p75	0.7834
p95	0.8837
Max	0.9394

7. Notes on interpretation

Nodule segmentation is severely class-imbalanced ($\sim 10^{-5}$ of voxels are nodule), which distorts the standard metric set:

- **Accuracy** is trivially near 1.0 for any reasonable model — the model gets ≈ 99.9649 % of voxels right by predicting "background" almost everywhere. Uninformative on its own here.
- **mIoU** averages the foreground and background IoU. `IoU_background` is essentially 1.0, so mIoU is roughly $0.5 + 0.5 \cdot \text{IoU_nodule}$. The useful signal is in `IoU_nodule` (0.4967) and Dice / F1 (0.6637).
- **Precision / Recall** are the standard per-voxel figures — no imbalance caveat needed. Recall 0.7818 means the model correctly labels ~ 78 % of nodule voxels; Precision 0.5766 means ~ 58 % of predicted-nodule voxels are true nodule.

Per-case Dice mean (0.5706) is lower than the micro Dice (0.6637) because large nodules dominate the micro average; per-case Dice weights each patient equally regardless of nodule size. Clinically the per-case distribution is the more useful summary.

8. How to load & run inference

Pre-trained weights are hosted at [Kakimaki00/nodule-segresnet-3d-wide-v2](#). Download and load directly:

```
huggingface-cli download Kakimaki00/nodule-segresnet-3d-wide-v2 --local-dir ./ckpt
```

```
import yaml, torch
from monai.networks.nets import SegResNet

cfg = yaml.safe_load(open("config.yaml"))["model"]
model = SegResNet(
    spatial_dims = cfg["spatial_dims"],
    in_channels   = cfg["in_channels"],
    out_channels  = cfg["out_channels"],          # 2
    init_filters  = cfg["init_filters"],
    blocks_down   = tuple(cfg["blocks_down"]),
    blocks_up     = tuple(cfg["blocks_up"]),
    dropout_prob  = cfg["dropout_prob"],
)
state = torch.load("model.pth", map_location="cpu", weights_only=True)
model.load_state_dict(state)
model.eval()

with torch.no_grad():
    # Input: (B, 1, 256, 256, 256), lung-bbox-cropped CT resampled to 256³.
    x = torch.randn(1, 1, 256, 256, 256)
    logits = model(x)                    # (B, 2, D, H, W)
    pred_class = logits.argmax(dim=1)    # (B, D, H, W) in {0, 1}
    nodule_mask = (pred_class == 1).to(torch.uint8)
```

This model expects **lung-bbox-cropped** input. A separate 2D ROI model is needed to produce that bbox — see the accompanying ROI checkpoints ([Kakimaki00/roi-segresnet-2d](#) or [Kakimaki00/roi-swinunetr-2d](#)), or use the demo's `inference.py` which chains them for you.

9. Reproducibility

- Config `configs/segresnet_wide_v2.yaml`
- Checkpoint `checkpoints/segresnet_wide_v2/best_model.pth`
- Metrics JSON `checkpoints/segresnet_wide_v2/eval_metrics_nodule_test.json`
- Command `python eval_metrics_nodule.py --config configs/segresnet_wide_v2.yaml --ckpt checkpoints/segresnet_wide_v2/best_model.pth --split test`

Companion two-stage nodule models on HuggingFace:

- [Kakimaki00/nodule-segresnet-3d-small](#) — small SegResNet on v2
- [Kakimaki00/nodule-dynunet-3d](#) — DynUNet (3D U-Net) on v2
- [Kakimaki00/nodule-segresnet-3d-wide-v1](#) — wide SegResNet on legacy v1 split